

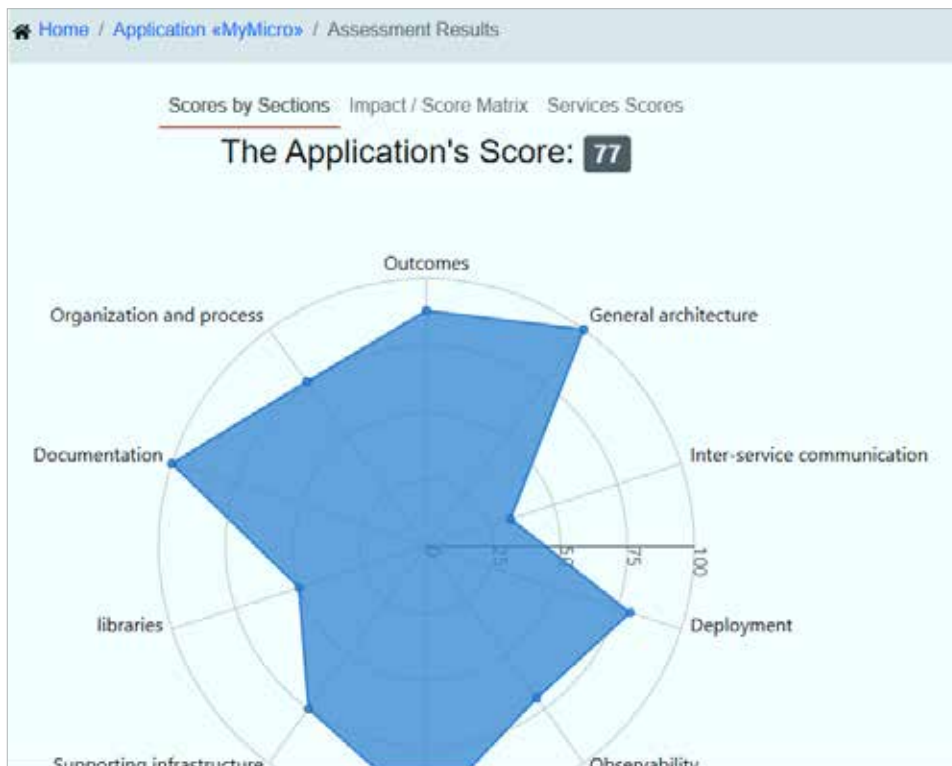


Figure 3: Survey-based qualitative assessment

of ownership and accountability. This autonomy can lead to increased productivity and innovation.

Example: At Google, teams are organised around microservices, enabling them to make decisions and implement changes without waiting for cross-team coordination.

Performance optimisation: Microservices can be optimised individually for performance, allowing for tailored solutions that meet specific service requirements. This can lead to improved response times and resource utilisation.

Example: LinkedIn uses microservices to optimise their messaging service, allowing them to handle millions of messages with minimal latency.

Overall system resilience: The distributed nature of microservices enhances system resilience. If one service encounters issues, the system can reroute requests or use fallback mechanisms to maintain functionality.

Example: During peak shopping seasons, companies like Walmart utilise microservices to ensure that their systems remain operational even under heavy load, employing strategies like circuit breakers to manage service failures.

Different ways to design microservices architecture

Developing microservices involves several design approaches that focus on creating independently deployable, scalable, and maintainable services. Here are the important ones.

Domain-driven design (DDD): This approach emphasises alignment between the software architecture and business domains. DDD involves identifying distinct domains within the business and developing microservices around them, ensuring that each service corresponds to a specific business context or sub-domain.

Decomposition by business capability: Instead of decomposing services by technical layers (e.g., MVC

layers), microservices are decomposed based on business capabilities. Each microservice is responsible for a discrete piece of business functionality and includes everything necessary to execute that functionality.

Database per service:

Each microservice owns its database schema or even a full database. This approach avoids database integration issues and ensures that microservices are loosely coupled. However, it introduces challenges related to data consistency and integration across services.

API gateway: A single entry point that handles requests and directs them to the appropriate microservices, the API

gateway can handle cross-cutting concerns such as authentication, logging, SSL termination, and rate limiting.

Event-driven architecture: In this model, microservices communicate through asynchronous events. This approach reduces coupling between services, enabling them to evolve independently. It also supports better scalability and responsiveness.

CQRS (command query responsibility segregation): This pattern separates the models for reading and writing data. It allows the system to scale better by optimising the read model for performance and the write model for data integrity and transactional safety.

Service discovery: In a microservices architecture, services need to dynamically discover and communicate with each other. Service discovery tools and mechanisms allow services to query for the network locations of other services.